

Lab 6 Submission Guidelines

EE 446: TinyML

Lab 6 has two parts and will be graded out of 100 points.

Submission method: Submit one public GitHub link on Canvas. The link must point directly to the Lab 6 folder. Include the public Edge Impulse project link inside Lab6_Report.pdf.

Privacy note: Do not include student ID numbers or UW NetIDs in the GitHub repository, report, or folder names. Canvas already records the submitter information.

Canvas Submission

Submit one public GitHub link on Canvas. The submitted link must point directly to the Lab 6 folder, not only to the repository homepage.

Required GitHub Folder Structure

Create a folder named:

```
Lab6_TinyML_Anomaly_Detection
```

The folder must contain:

```
Lab6_TinyML_Anomaly_Detection/
|-- Lab6_Report.pdf
|-- TinyML_Lab6.ipynb
`-- Activity_Anomaly/
    |-- Activity_Anomaly.ino
    `-- autoencoder_model.cc
```

Submission Components

Component	Points	Required submission evidence
Part 1: Edge Impulse Integration and Deployment	50	Public Edge Impulse project link included in Lab6_Report.pdf; answers to required questions; deployment screenshot and interpretation.
Part 2: Quantized Autoencoder-Based Activity Anomaly Detection	50	Completed notebook, Arduino deployment files, model file, screenshots, evaluation results, and discussion answers.

Part 1: Edge Impulse Integration and Deployment (50 points)

Complete the Edge Impulse fan anomaly detection workflow. This includes cloning the public Edge Impulse project, reviewing the impulse design, training the classifier, training the anomaly detector, and deploying the model to the Arduino Nano 33 BLE Sense.

Important: The public Edge Impulse project link must be included inside Lab6_Report.pdf.

Required Part 1 Content in Lab6_Report.pdf

1. Raw inputs: State the raw inputs used in the Edge Impulse project.
2. NN classifier features: State the type of features passed into the NN classifier, the number of input features, and at least five actual feature names used to train the classifier.
3. NN classifier outputs: State the outputs of the NN classifier.

Submit the public GitHub Lab 6 folder link through Canvas.

4. Anomaly detector features: State the type of features passed into the anomaly detector, the number of features used, and the feature names used to train the anomaly detector.
5. Deployment evidence: Include a screenshot showing the Edge Impulse model running on the Arduino Nano 33 BLE Sense and producing inference results.
6. Short interpretation: Briefly interpret the observed deployment results. Discuss whether the classifier output should always be trusted and under what conditions it may or may not be reliable.

Part 2: Quantized Autoencoder-Based Activity Anomaly Detection (50 points)

Complete the Lab 6 notebook using the tinymL-arduino Python environment set up for this class. The notebook trains and evaluates a quantization-aware autoencoder, converts the model to an int8 TensorFlow Lite model, and generates the model source file for Arduino deployment.

Required Part 2 Files

- TinyML_Lab6.ipynb: completed notebook with required code cells and generated outputs.
- Activity_Anomaly/Activity_Anomaly.ino: Arduino sketch used for deployment.
- Activity_Anomaly/autoencoder_model.cc: generated int8 model source file used by the Arduino sketch.

Required Content in Lab6_Report.pdf

Submit one concise PDF report named Lab6_Report.pdf. The report must include results from both parts of the lab and should be clearly organized using the sections below.

Section 1: Edge Impulse Results

- Public Edge Impulse project link
- Answers to the Part 1 questions
- Screenshot of successful Arduino deployment/inference
- Short interpretation of the observed results

Section 2: Autoencoder Model Results

Include clear and labeled screenshots of the following:

- Training and validation loss curves
- PCA visualization
- t-SNE visualization
- UMAP visualization
- Reconstruction error distribution
- Selected anomaly threshold
- Confusion matrix
- Classification report or evaluation metrics
- Arduino serial monitor output showing reconstruction error and normal/anomaly detection results

Section 3: Discussion Questions

Answer the following briefly:

- What threshold did you choose for anomaly detection, and why?
- How does reconstruction error help distinguish normal and anomalous activity?
- What information from the Python notebook must be preserved when deploying the model to Arduino?
- Why is it important for the Arduino sketch to use the same preprocessing assumptions as the notebook?
- What are the main limitations of this anomaly detection method when deployed on a microcontroller?

Final Submission Checklist

- The GitHub link is public and accessible.

- The GitHub link points directly to the Lab 6 folder.
- Lab6_Report.pdf is included in the Lab 6 folder.
- The public Edge Impulse project link is included in Lab6_Report.pdf.
- TinyML_Lab6.ipynb is completed and included.
- Activity_Anomaly.ino and autoencoder_model.cc are included inside the Activity_Anomaly folder.
- The report contains the required screenshots and discussion answers for both parts.
- No student ID numbers or UW NetIDs are included in the public repository or report.